Project Work Parallel Programming: Acceleration of Quantum Kernel Methods for Image Classification Tasks

Fouepe Dylan
dylan.fouepe@edu.unifi.it

University of Florence

June 16, 2026

Abstract

This project studies binary SVM training on classical image datasets using simulated quantum kernels, with a strong focus on parallel programming and heterogeneous CPU/GPU optimization. The project repository provides three execution paths: a classical CPU baseline, a GPU quantum backend based on PennyLane+PyTorch (**GPU base**), and a custom HPC backend (**GPU custom**) built on CuPy, DLPack, and a bespoke C++ CUDA kernel. The core of the work is to build quantum Gram matrices of size $N \times N$ quickly and correctly, whose cost remains $O(N^2 \cdot 2^Q)$ for Q qubits.

Experiments on Fashion-MNIST, CIFAR-10, and SVHN reveal two regimes. On Fashion-MNIST, the quantum kernel remains strong in the latest run-scoped campaign, with values up to **F1 = 0.9653** and **AUC = 0.9956** (**GPU base**, size 1000). On CIFAR-10 and SVHN, however, kernels tend to concentrate, the inter-class gap becomes nearly zero, and the SVM often ends with a support-vector fraction close to 1.0, consistent with a *kernel collapse* or *barren-plateau-like* behavior. Overall, the reported results provide a consistent comparison of throughput and predictive quality across classical and quantum-kernel settings.

1 Introduction

Quantum kernel methods provide a clean way to project classical data into a very high-dimensional Hilbert space and then solve a classification task with a precomputed SVM [1]. In practice, this promise quickly runs into a cost bottleneck: for N samples and Q qubits, building the Gram matrix requires simulating states of dimension 2^Q and evaluating $O(N^2)$ complex overlaps. At $Q = 16$ a single state vector already contains 65,536 complex amplitudes.

The study compares a classical SVM baseline and two quantum-kernel backends under the same experimental protocol. The report focuses on comparative evidence: predictive quality (F1/AUC), runtime, throughput, and failure indicators related to kernel concentration.

2 Scope of the Project

This study addresses three questions:

- How do classical and quantum-kernel SVMs compare on the same binary tasks?
- Which backend provides the best throughput/runtime trade-off at 16 qubits?
- Which empirical signals indicate kernel concentration and reduced separability?

The analysis focuses on measurable outcomes: F1, AUC, runtime, throughput, and kernel-shape diagnostics.

3 Experimental Methodology

Three datasets are used for binary classification: Fashion-MNIST [2], CIFAR-10 [3], and SVHN [4]. Each is binarized into three tasks of increasing difficulty (see Table 1).

Hardware Setup: All benchmarks and training pipelines were executed on a dedicated HPC node equipped with an AMD EPYC 7343 16-Core Processor and a NVIDIA RTX 6000 Ada Generation GPU (48 GB VRAM). The environment relies on CUDA 13.1, providing massive parallelism for both the native PyTorch and custom CUDA backends. All experiments use $Q = 16$ qubits by default, yielding state vectors of dimension $2^{16} = 65{,}536$ complex amplitudes. Key performance observations from the benchmark campaign at this scale are:

- at 16 qubits, the direct backend comparison places **GPU custom** ahead of both **CPU base** and **GPU base** in throughput;
- across the measured 16-qubit configurations, peak GPU memory remains in a moderate range (about 0.06 GB to

Table 1: Binary classification tasks configured in the repository.

Dataset	Easy	Med	Hard
Fashion-MNIST	0 vs 1	3 vs 8	0 vs 6
CIFAR-10	1 vs 6	0 vs 2	3 vs 5
SVHN	1 vs 0	2 vs 7	3 vs 5

3.00 GB), indicating that throughput gains are not tied to extreme VRAM pressure.

From the quantum perspective, most configurations use 16 PCA components (thus 16 qubits) with a shallow circuit depth by default. Circuit layers are configurable via YAML through *layer config*; an important case is a hard CIFAR-10 configuration, which forces **6 layers** and serves as a "hard" regime to observe kernel concentration.

3.1 Preprocessing and Quantum Feature Map

The training pipeline follows these steps:

1. extract pixel values and flatten;
2. apply PCA to $d = 16$ components;
3. scale features via *range scaling*;
4. encode features using the *angle map*, *Y map* or *YZ map*;
5. simulate states via PennyLane/*state simulator*.

The experiments use the **YZ ansatz** with entangling layers, which provides a simple trade-off between expressivity and simulation cost. Circuit weights are initialized once per run from a normal distribution and kept fixed during SVM training; this is therefore not a full variational training loop but a *kernel-based* learning approach.

The target kernel is the fidelity kernel

$$k(x_i, x_j) = \left| \langle \psi(x_i) \mid \psi(x_j) \rangle \right|^2 .$$

In practice, if $S_A \in \mathbb{C}^{N_A \times 2^Q}$ and $S_B \in \mathbb{C}^{N_B \times 2^Q}$ are the state matrices, then

$$K_{ij} = \left| \sum_{k=1}^{2^Q} S_A[i,k] \, \overline{S_B[j,k]} \right|^2 .$$

3.2 Training Protocol

The experimental protocol uses:

- train/test split inherited from the dataset;
- 3-fold cross-validation on the training partition;

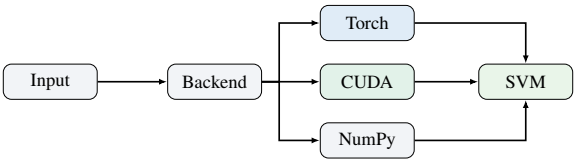


Figure 1: Unified backend API: the Gram matrix is computed on GPU (**GPU base** or **GPU custom**) or CPU (**CPU base**); the SVM solver always runs on CPU.

- hyperparameter optimization of C via Optuna [5] (sampling C over a log-uniform distribution);
- final re-fit on train+val;
- final evaluation on the test set.

In addition to F1 and AUC, the analysis tracks runtime, support-vector fraction, and kernel distribution statistics to distinguish optimization effects from representational limits.

4 HPC Architecture

4.1 Unified Backend API

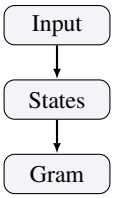
For Gram computation, we use a unified API that accepts the same data and normalization flags, but delegates execution to three paths:

- **CPU base**: CPU fallback, useful as a reference and for small cases;
- **GPU base**: generate states and perform matrix computation directly on the GPU with PyTorch;
- **GPU custom**: generate states with PyTorch, convert tensors to CuPy via DLPack (enabling efficient zero-copy memory sharing without CPU round-trips), and compute the Gram with a custom *CUDA kernel*.

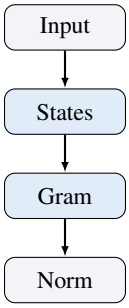
Figure 1 summarises the delegation flow from preprocessed features to the SVM solver.

4.2 NumPy CPU Backend

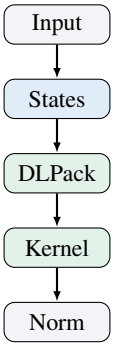
The **CPU base** backend runs Gram-matrix computation on CPU and serves as the baseline path for backend comparison. It is used as a numerical reference and remains practical for smaller kernels where CPU overhead is limited. Unlike the GPU backends, it does not rely on VRAM-aware tiling, stream orchestration, or autotuning. Figure 2a illustrates its computation flow.



(a) CPU base flow.



(b) GPU base flow.



(c) GPU custom flow.

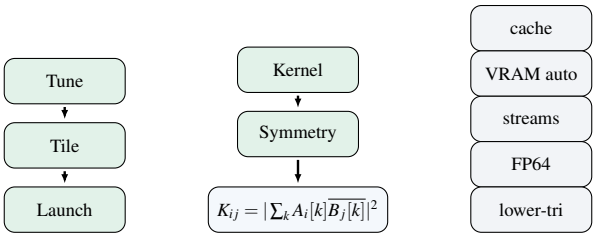
Figure 2: Comparison of backend pipelines

4.3 Torch GPU Backend

The **GPU base** backend uses native PyTorch GPU operations to generate states and compute Gram matrices. In this study, it is treated as the main GPU reference point for quality/runtime comparison against both **CPU base** and **GPU custom**. Its distinctive optimizations are pinned host memory and CUDA streams, which target transfer overlap rather than custom kernel tuning. Figure 2b shows its internal flow.

4.4 Custom CUDA Backend GPU custom

The **GPU custom** backend computes Gram matrices with custom CUDA kernels through CuPy. In this report, its role is assessed through measured throughput and downstream classification metrics, not by internal kernel structure. Figure 2c details its computation pipeline.



(a) Host flow. (b) Device core. (c) Optimization.

Figure 3: Kernel construction in **GPU custom**, organized across three elements: host orchestration, device kernel core, and optimization controls.

4.4.1 Backend Optimizations

Backend-specific optimizations are:

- **GPU base path**: pinned host memory and CUDA streams are used to overlap transfer/compute stages, without custom CUDA kernel tuning.
- **VRAM-aware sizing**: state_tile=-1 automatically sizes state block dimensions according to available VRAM.
- **Autotuning**: CUDA kernel tile sizes are learned and cached persistently across runs. A specific entry for 16 qubits in double precision is handled.
- **Bulk precompute**: when memory permits, all states in a block are prebuilt to reduce round-trips and leverage GPU parallelism.
- **Stream pool**: multiple CUDA streams can be used to orchestrate state preparation and Gram computation.
- **Memory profiling**: the benchmark measures VRAM usage, transfers and throughput.
- **CPU base path**: Use smal state_tile sizes to take advantage of CPU cache and avoid VRAM-like bottlenecks. The CPU path does not use pinned memory or streams, as these are GPU-specific optimizations. Instead, it relies on efficient NumPy operations and multi-threading where possible.

Benchmark ablations identify the most useful settings:

- disabling precompute is the slowest option in the reported **GPU custom** ablation rows;
- vram_fraction=0.95 is the selected optimum in each of the stored profile logs;
- num_streams=1 is the selected optimum in the stored stream-sweep rows;
- torch_pinned+streams is the best torch configuration in the stored torch ablation rows.

Table 2: Sample-scaling throughput (Mpairs/s) by dataset and backend from per-dataset benchmark CSVs. Values are reported at the highest sample-scaling qubit setting available in each dataset profile (Fashion: 12 qubits; CIFAR-10 and SVHN: 16 qubits).

Dataset	Size	Qubits	CPU base	GPU base	GPU custom
fashion	256	12	0.0059	0.0140	0.0159
fashion	512	12	0.0224	0.0299	0.0298
fashion	1024	12	0.0856	0.0575	0.0546
cifar10	256	16	0.0020	0.0057	0.0062
cifar10	512	16	0.0086	0.0175	0.0112
cifar10	1024	16	0.0319	0.0375	0.0245
svhn	256	16	0.0023	0.0053	0.0058
svhn	512	16	0.0097	0.0175	0.0124
svhn	1024	16	0.0328	0.0366	0.0243

At this scale, data movement and memory organization have a larger effect than increasing stream complexity.

4.4.2 Normalization and Metric Comparability

All backends share the same post-processing and kernel normalization protocol so that F1/AUC and runtime differences can be interpreted as backend effects rather than normalization artifacts:

- finite-value checks (NaN/Inf) before writing kernel blocks;
- diagonal validation before normalization and classifier fitting;
- shared kernel normalization across backends;
- strict cross-normalization

$$\hat{K}(X,Y)_{ij} = \frac{K(X,Y)_{ij}}{\sqrt{K(X,X)_{ii}K(Y,Y)_{jj}}}$$

  applied after computing both diagonals;
- identical post-normalization path before SVM fitting and evaluation.

This shared normalization pipeline is the basis for fair backend-to-backend metric comparisons.

5 Performance Results

5.1 Benchmark

The benchmark captures qubit scaling, sample scaling, optimization ablations, tile-state tuning, VRAM-fraction sensitivity, stream-count impact, and kernel fidelity against the **CPU base** reference. The validations are run across the datasets with two warmup runs and two timed runs per point. Outputs include per-dataset benchmark summaries, figures, and execution logs.

Key observations from these benchmark statistics are:

- at small sample sizes (256), GPU backends are consistently faster than NumPy across datasets;
- at 512 samples, **GPU base** is the fastest backend in all three dataset profiles;
- at 1024 samples, **GPU base** remains fastest on CIFAR-10 and SVHN, while **CPU base** is fastest on Fashion.

The benchmark plots in Figs. 4, 5, and 6 are consistent with Table 2: backend ranking depends on both dataset profile and sample size.

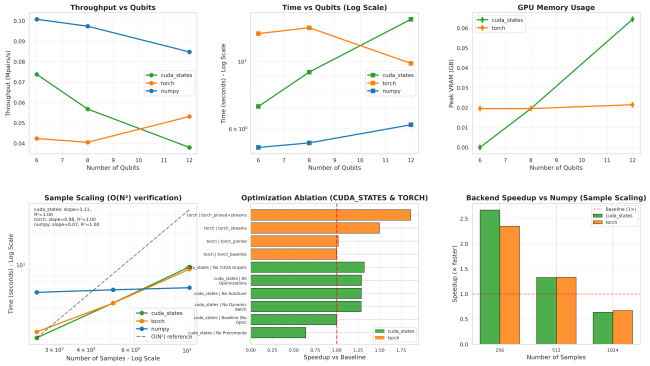


Figure 4: Fashion-MNIST benchmark profile.

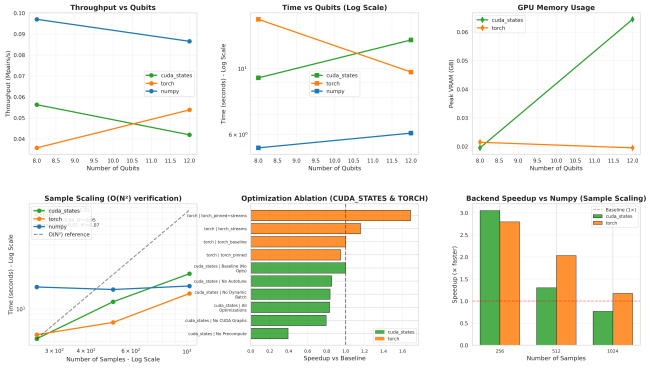


Figure 5: CIFAR-10 benchmark profile.

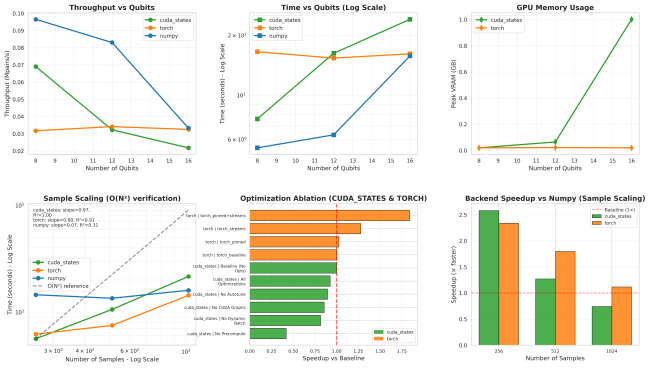


Figure 6: SVHN benchmark profile.

6 Classification Results

Table 3 shows that, within quantum backends, **GPU base** is generally faster and often more accurate than **GPU custom**, while both remain strong on Fashion and weak on SVHN. Note that the reported **Mean time** measures the full pipeline execution, including state simulation, Gram matrix building, and the complete Optuna cross-validation loop.

6.1 Quantum Comparison Against the Classical Baseline

The comparison is performed on the shared setup (same dataset, same difficulty, and same sample sizes 500/1000).

The paired analysis is now stratified by sample size. Table 4 reports size-aware deltas.

Across both sizes, classical remains dominant in paired wins and runtime, while the size-wise split shows the gap is larger at 1000 samples than at 500.

7 Failure Analysis: Kernel Collapse and Barren Plateau-Like Regimes

Hard tasks exhibit the following quantitative pattern:

- inter-class gap near zero after centering/normalization;
- test-kernel standard deviation on the order of 10^{-4} to 10^{-3};
- support-vector fraction near 1.0;
- AUC close to 0.5 (random guessing) in the hardest cases.

The hard CIFAR-10 case with **6 layers** is particularly revealing. In this regime, one typically observes:

- *kernel std = 0.0005*;
- *SV frac = 1.0000*;
- *test F1 = 0.0000*;
- *test AUC = 0.4757*.

Table 3: Quantum backend performance by dataset and sample size (mean over difficulty levels in the latest run-scoped extraction).

Dataset	Size	Backend	Mean F1	Mean AUC	Mean time (s)
cifar10	500	**GPU custom**	0.3500	0.4480	336.13
cifar10	500	**GPU base**	0.4270	0.4812	190.61
cifar10	1000	**GPU custom**	0.4305	0.4365	496.73
cifar10	1000	**GPU base**	0.5729	0.5532	302.19
fashion	500	**GPU custom**	0.8905	0.9569	237.05
fashion	500	**GPU base**	0.8889	0.9574	110.78
fashion	1000	**GPU custom**	0.8958	0.9553	339.29
fashion	1000	**GPU base**	0.8998	0.9557	187.13
svhn	500	**GPU custom**	0.1789	0.4719	545.39
svhn	500	**GPU base**	0.1547	0.4719	302.73
svhn	1000	**GPU custom**	0.1596	0.4846	637.30
svhn	1000	**GPU base**	0.2998	0.4846	380.61

Table 4: Run-scoped paired comparison by sample size. Positive deltas indicate better classical quality; negative time deltas indicate faster classical execution.

Comparison	Size	ΔF1	ΔAUC	ΔTime (s)	Wins C/B/T
Classical − GPU base	500	+0.1708	+0.1488	-188.28	8/0/1
Classical − GPU base	1000	+0.1721	+0.1600	-274.52	8/1/0
Classical − GPU custom	500	+0.1879	+0.1600	-359.76	8/0/1
Classical − GPU custom	1000	+0.2676	+0.1990	-475.66	8/1/0

This behavior is consistent with a **kernel concentration**: states become almost indistinguishable in terms of overlap used by the SVM. It is reasonable to interpret this as a *barren-plateau-like* or *kernel collapse* regime [6, 7], although multiple factors likely combine:

- loss of information due to conversion to grayscale and subsequent PCA;
- generic ansatz not informed by image geometry;
- increased depth favoring concentration of overlaps;
- intrinsic limitations of global fidelity kernels on complex visual data.

These observations indicate that acceleration alone does not remove the representational limits of the feature map.

8 Data and Code Availability

Project repository: **github.com/DylanUnifi**.

Datasets used in this study:

- **Fashion-MNIST** [2].
- **CIFAR-10** [3].
- **SVHN** [4].

Table 5: Difficulty-level summary across quantum backends (mean over datasets and sizes, latest run-scoped extraction).

Diff./Backend	F1	AUC	Time (s)
easy / GPU custom	0.4907	0.6327	475.44
easy / GPU base	0.6366	0.6499	258.76
med / GPU custom	0.5263	0.6314	404.06
med / GPU base	0.6560	0.6896	226.39
hard / GPU custom	0.4357	0.6125	416.44
hard / GPU base	0.3289	0.6126	251.88

9 Conclusion

The updated benchmark and classification tables show that backend ranking is not global: it depends on sample size, dataset profile, and backend implementation details. In the benchmark campaign, GPU paths dominate low-size regimes, while some larger-size points still favor **CPU base** in specific profiles. On predictive quality, both quantum backends remain competitive on Fashion-MNIST but degrade on CIFAR-10 and especially SVHN, where collapse indicators and near-random AUC values appear repeatedly in hard settings. Overall, acceleration improves throughput but does not remove the representational bottleneck of the current feature map/ansatz pair on harder visual distributions.

10 Perspectives

Data-driven next steps are:

- projected or local kernels to reduce global fidelity concentration effects;
- geometry-aware ansatz design tailored to image structure;
- stronger classical feature front-ends before PCA and quantum embedding;
- broader benchmark grids at fixed qubit counts per dataset;
- multi-GPU or MPI strategies to distribute one large Gram matrix build;
- native GPU-side SVM solvers to remove the final CPU training bottleneck;
- richer diagnostics (gap trajectories, spectra, support-vector dynamics) for earlier collapse detection.

References

[1] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta, "Supervised learning with quantum-enhanced feature spaces," *Nature*, vol. 567, no. 7747, pp. 209–212, 2019.

[2] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.

[3] A. Krizhevsky, "Learning multiple layers of features from tiny images," tech. rep., University of Toronto, 2009.

[4] Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, and A. Y. Ng, "Reading digits in natural images with unsupervised feature learning," in *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*, 2011.

[5] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2623–2631, 2019.

[6] M. Larocca, S. Thanasilp, S. Wang, K. Sharma, J. Biamonte, P. J. Coles, L. Cincio, J. R. McClean, Z. Holmes, and M. Cerezo, "A review of barren plateaus in variational quantum computing," *Nature Reviews Physics*, vol. 7, pp. 174–189, 2025.

[7] S. Thanasilp, S. Wang, M. Cerezo, and Z. Holmes, "Exponential concentration and untrainability in quantum kernel methods," *Nature Communications*, vol. 15, p. 5200, 2024.